Conteúdo para Revisão

Parte 1: PHP e Formulários

A interação entre o usuário e o servidor ocorre primordialmente através de formulários. O foco é como o PHP recebe, estrutura e limpa esses dados.

1.1. Manipulação de Arrays

Arrays são as estruturas de dados mais versáteis no PHP. É essencial dominar as funções que os manipulam, especialmente arrays associativos.

- **Adicionar e Remover:**
  - **array_push($array, $valor)** ou **$array[] = $valor**: Adiciona um elemento ao final do array.
  - **array_pop($array):** Remove e retorna o último elemento.
  - **unset($array[$chave]):** Remove um elemento específico baseado no seu índice ou chave.
- **Ordenação:**
  - **sort()** / **rsort()**: Ordena arrays indexados em ordem crescente / decrescente (perde as chaves originais).
  - **asort()** / **arsort()**: Ordena arrays associativos pelo *valor*, mantendo a relação chave-valor.
  - **ksort()** / **krsort()**: Ordena arrays associativos pela *chave*.
- **Busca e Verificação:**
  - **in_array($valor, $array)**: Verifica se um valor existe no array (retorna booleano).
  - **array_key_exists($chave, $array)**: Verifica se uma chave específica existe.
  - **array_search($valor, $array)**: Busca um valor e retorna a sua chave correspondente.

1.2. Criação de Formulários Dinâmicos

Elementos HTML como <select>, <input type="radio"> ou <input type="checkbox"> não devem ser escritos manualmente quando as opções vêm do servidor.

- Utilizar a estrutura de repetição foreach para iterar sobre um array de categorias ou produtos e gerar as tags HTML dinamicamente.
- Exemplo:

```
<select name="estado_selecionado" id="estado">
    <option value="">-- Selecione --</option>
    <?php
        foreach ($estados_brasileiros as $sigla => $nome) {
            echo "<option value='{$sigla}'>{$nome}</option>";
        }
    ?>
</select>
```

1.3. Validação e Sanitização (Filtros)

Nunca se deve confiar nos dados enviados pelo usuário via $_POST ou $_GET.

- **A função filter_input():** É a maneira mais segura de resgatar dados, substituindo o acesso direto às superglobais.
- **Sanitização (Limpeza):** Remove caracteres indesejados. Ex: FILTER_SANITIZE_SPECIAL_CHARS converte tags HTML (<script>) em entidades inofensivas, prevenindo ataques XSS.
- **Validação (Verificação):** Checa se o dado obedece a um formato. Ex: FILTER_VALIDATE_EMAIL ou FILTER_VALIDATE_INT. Se o formato for inválido, a função retorna false.

Parte 2: Sessões e Controle de Acesso

O protocolo HTTP não possui estado (é *stateless*). As sessões ($_SESSION) são a memória do servidor, permitindo que informações "viajem" de uma página para outra enquanto o navegador estiver aberto.

2.1. Ciclo de Vida da Sessão

- **Inicialização:** A instrução session_start() deve ser obrigatoriamente a primeira linha de código lógico do script, antes de qualquer saída de texto ou HTML.
- **Armazenamento:** A superglobal $_SESSION funciona como um array associativo padrão. Ex: $_SESSION['usuario'] = 'Maria';.
- **Destruição:** Para realizar o *logout* do sistema, utiliza-se session_destroy() e a limpeza do array com $_SESSION = [];. *Além da melhor prática de limpeza do cookie de sessão.

2.2. Autenticação vs. Autorização

É fundamental distinguir e aplicar ambos os conceitos no controle de acesso:

- **Autenticação (Quem é você?):** O processo de verificar as credenciais no login. Se o e-mail e senha estiverem corretos, criamos a variável que "assina" o crachá do usuário (ex: $_SESSION['logado'] = true).
- **Autorização (O que você pode fazer?):** O processo de verificar o nível de permissão. Em páginas restritas, logo após o session_start(), verifica-se a sessão e o papel do usuário (ex: $_SESSION[perfil] === “admin”).

2.3. Personalização de Interface via Controle de Acesso

A renderização do HTML deve ser condicional, baseada nos dados contidos na sessão atual:

- **Exemplo Prático (Admin vs. Usuário Normal):**

```php
if ($_SESSION['nivel_acesso'] === 'administrador') {
    // Exibe menus de gerenciamento completos, exclusão de usuários e relatórios gerais.
    echo "<button>Excluir Registros</button>";
} else {
    // Exibe apenas visão do próprio usuário comum, limitando ações críticas.
    echo "<p>Bem-vindo ao seu painel restrito.</p>";
}
```

Parte 3: Simulação de Banco de Dados (CRUD em Arrays)

Para testar a lógica de negócios sem a complexidade de um SGBD real (como MySQL), utilizaremos arrays multidimensionais para simular tabelas de banco de dados.

Importante: Para que essas modificações persistam ao recarregar a página (ao enviar um formulário), esse "banco de dados" simulado deve ser armazenado dentro de uma variável de sessão.

3.1. Estrutura do Banco Simulado

A tabela é um array indexado, onde cada "linha" é um array associativo contendo os campos.

```php
if (!isset($_SESSION['banco_usuarios'])) {
    $_SESSION['banco_usuarios'] = [
        ['id' => 1, 'nome' => 'Ana', 'email' => 'ana@email.com'],
        ['id' => 2, 'nome' => 'Bruno', 'email' => 'bruno@email.com']
    ];
}
```

3.2. Operações CRUD Lógicas

- **Create (Inserção):** Receber dados higienizados do formulário, gerar um novo ID e adicionar a nova linha com array_push() ou $_SESSION['banco_usuarios'][] = $novo_registro.
- **Read (Leitura):** Utilizar um foreach para percorrer $_SESSION['banco_usuarios'] e montar uma tabela HTML (<table>, <tr>, <td>) listando os registros atuais.
- **Update (Atualização):** Buscar o índice do registro específico (utilizando o ID como referência), resgatar os dados para preencher o formulário de edição e, ao salvar, sobrescrever os valores naquele índice exato.
- **Delete (Exclusão):** Identificar o índice no array correspondente ao ID a ser excluído e aplicar a função unset(). Após a exclusão, recomenda-se usar array_values() para reordenar os índices numéricos do array principal e evitar lacunas.